SAD — Software Architecture Document

Mini Project 1: Product Information System (Desain)

Atribut	Keterangan
Mata Kuliah	Pemrograman Web — Pertemuan 2
Bahasa / Platform	PHP (native), HTML, CSS
Jenis Arsitektur	Layered Architecture (3 Layer)
Tujuan	Merancang blueprint sistem manajemen data informasi produk siap pakai

---

1. Pendahuluan

1.1 Tujuan

Dokumen ini mendeskripsikan rancangan arsitektur konseptual sistem informasi produk: bagaimana data disimpan, diolah, dan ditampilkan, serta hubungan antar-komponennya.

1.2 Ruang Lingkup

- Menyimpan data produk (ID, Nama, Kategori, Harga, Stok, Deskripsi).
- Menghitung total nilai aset stok gudang.
- Menyorot baris tabel produk dengan **stok kritis (< 3)**.
- Menampilkan seluruh data dalam tabel HTML.

1.3 Di Luar Lingkup

Database (MySQL), autentikasi, CRUD lewat form, dan API.
Data masih berupa *array* statis (blueprint tahap awal).

---

2. Gambaran Arsitektur

```
flowchart TB
    subgraph PRES["3. PRESENTATION LAYER — index.php"]
        A1["require_once products.php & functions.php"]
        A2["foreach: render baris tabel HTML"]
        A3["Tampilkan total nilai stok"]
    end
    subgraph PROC["2. PROCESSING LAYER — functions.php"]
        B1["hitungTotalNilaiStok()"]
        B2["Conditional: stok < 3 → class 'stok-kritis'"]
        B3["formatRupiah()"]
    end
    subgraph DATA["1. DATA LAYER — products.php"]
        C1["$products = array multidimensi (associative)"]
    end
    PRES -->|memanggil fungsi| PROC
    PRES -->|membaca data| DATA
    PROC -->|menerima array sebagai parameter| DATA
```

Alur ketergantungan: **Presentation → Processing → Data**. Layer bawah tidak mengetahui layer di atasnya (*low coupling*).

---

3. Komponen Arsitektur

3.1 Data Layer — `products.php`

Aspek	Rancangan
Tanggung jawab	Menampung seluruh data produk
Struktur	*Multidimensional array*: array indexed berisi *associative array* per produk
Field	`id`, `nama`, `kategori`, `harga`, `stok`, `deskripsi`
Aturan	Hanya berisi data; **tanpa logika & tanpa HTML**

Contoh struktur satu record:

```
[
    'id'        => 'P001',
    'nama'      => 'Laptop Asus Vivobook 14',
    'kategori'  => 'Elektronik',
    'harga'     => 8500000,
    'stok'      => 10,
    'deskripsi' => '...'
]
```

3.2 Processing Layer — `functions.php`

Fungsi	Input	Output	Keterangan
`hitungTotalNilaiStok(array $products)`	array produk	`int/float`	Σ (harga × stok) seluruh produk
`isStokKritis(int $stok)`	stok	`bool`	`true` bila stok < 3 (konstanta `BATAS_STOK_KRITIS`)
`getKelasBaris(int $stok)`	stok	`string`	Mengembalikan `'stok-kritis'` atau `''` (logika conditional penyaring warna baris)
`formatRupiah(int/float $angka)`	angka	`string`	Format `Rp 1.000.000`

Aturan: fungsi bersifat **murni (pure function)** – tidak membaca variabel global, tidak melakukan `echo`.

3.3 Presentation Layer — `index.php`

- Merajut komponen memakai `require_once`.
- Merender data ke tabel HTML dengan perulangan `foreach`.
- Menambahkan class CSS dari hasil `getKelasBaris()` pada tiap `<tr>`.
- Menampilkan total nilai stok di `<tfoot>`.
- Semua output diamankan dengan `htmlspecialchars()`.

---

4. Struktur Direktori

```
product-info-system/
├── index.php        # Presentation Layer
├── products.php     # Data Layer
├── functions.php    # Processing Layer
├── SAD.md
├── Processing_flowchart.md
└── README.md
```

---

5. Keputusan Desain (Design Rationale)

5.1 Mengapa *associative array*, bukan *indexed array*?
Akses `$p['harga']` bersifat self-documenting, sedangkan `$p[3]` bergantung pada urutan posisi. Jika urutan kolom berubah, indexed array diam-diam menghasilkan data salah (mis. harga terbaca sebagai stok) tanpa error. Associative array juga lebih aman ditambah field baru tanpa menggeser index.

5.2 Mengapa `require_once`, bukan `include`?

- `require` → jika file gagal dimuat terjadi **Fatal Error** dan eksekusi berhenti.
- `include` → hanya **Warning**, skrip lanjut berjalan. Untuk file vital (koneksi database/data), ini berbahaya: aplikasi berjalan tanpa koneksi, menghasilkan halaman kosong/rusak, dan error message dapat membocorkan path server.
- `_once` mencegah pemuatan ganda yang menyebabkan *redeclare function* error.

5.3 Mengapa *logic error* lebih sulit dilacak dibanding *syntax error*?
Syntax error dideteksi parser secara langsung dengan pesan dan nomor baris. Logic error tidak menghasilkan pesan apa pun — program berjalan "normal" tetapi hasilnya salah (mis. `stok <= 3` padahal seharusnya `< 3`). Pelacakan menuntut pengujian dengan data uji (*test case*) dan pengecekan manual. Karena itu logika dipisah dalam fungsi kecil agar mudah diuji satu per satu.

5.4 Risiko struktural *array* multidimensi dengan nesting terlalu dalam

- Penggunaan memori membengkak (setiap level array menambah overhead *hash table*), dan PHP menyalin array saat dimodifikasi (*copy-on-write*).
- Kompleksitas iterasi meningkat (nested `foreach` → O(n·m…)).
- Kode sulit dibaca/dirawat, rawan error `Undefined index`.
- Menyalin struktur relasional database ke array bersarang tidak skalabel; pada data besar lebih tepat memakai query database, *pagination*, atau *generator*.

---

6. Kendala & Aturan Mutu (Non-Functional)

Kategori	Aturan
Separation of Concerns	Data, logika, dan tampilan berada di file terpisah
Keamanan	`htmlspecialchars()` untuk seluruh output dinamis
Maintainability	Batas stok kritis dijadikan konstanta
Reliabilitas	Memakai `require_once` untuk dependensi wajib
Keterbacaan	Penamaan konsisten, komentar pada fungsi

7. Pengembangan Lanjutan

Migrasi data layer ke MySQL (PDO), fitur CRUD, pencarian & filter kategori, serta pagination.